

PARAKH — Stage 1 PRD

Data Ingestion & Schema Layer

1. Objective

Build a **robust, schema-aware data ingestion pipeline** that:

1. Accepts raw tabular data (CSV/Parquet)
2. Validates structure and data types
3. Cleans invalid or placeholder values
4. Produces a **normalized, strongly-typed dataset (Corpus)**

This stage must work **even when no real dataset is provided**, by generating **synthetic but realistic data** aligned with MPLADS-like schemes.

2. Scope

Included

- Synthetic dataset generation (mandatory)
- File ingestion (CSV + Parquet)
- Schema validation
- Data cleaning & normalization
- Basic anomaly flags (invalid dates, nulls, etc.)
- Unified in-memory representation (Corpus)

Excluded

- Confidence scoring
 - Risk modeling
 - Clustering or ML
 - External APIs
-

3. Functional Requirements

3.1 Synthetic Data Generation (CRITICAL)

If no dataset is provided, the system must generate one.

Dataset characteristics:

- Size: **10,000 – 50,000 records**
- Domain: public works (roads, schools, drainage, etc.)

Required fields:

Field	Type	Description
work_id	string	Unique identifier
work_name	string	Free text description
district	string	District name
state	string	State name
sanction_amount	float	Approved cost
amount_spent	float	Actual expenditure
date_proposal	date	Proposal date
date_approval	date	Approval date
date_completion	date	Completion date
implementing_agency	string	Agency name
vendor_name	string	Contractor/vendor
status	string	{proposed, approved, completed}

3.2 Synthetic Data Properties

The generated dataset must include:

☒ Normal records

- Logical date order
- Reasonable cost ranges

Imperfect records (IMPORTANT)

Inject realistic noise:

- 10–20% missing fields
- 5–10% invalid date ordering
- 5% extreme cost outliers
- 5% duplicate/near-duplicate work names
- Placeholder values:
 - "N/A", "unknown", "0000-00-00"

Goal:

3.3 Data Ingestion

Support:

```
Corpus.from_csv(path)
Corpus.from_parquet(path)
Corpus.from_dataframe(df)
```

Requirements:

- Automatic type inference
 - Graceful failure on malformed rows
 - Logging of ingestion errors
-

3.4 Schema Definition

Define a strict schema:

```
Schema = {
    "work_id": str,
    "work_name": str,
    "district": str,
    "state": str,
    "sanction_amount": float,
    "amount_spent": float,
    "date_proposal": datetime,
    "date_approval": datetime,
    "date_completion": datetime,
    "implementing_agency": str,
    "vendor_name": str,
    "status": str,
}
```

3.5 Validation Rules

Each record must be validated:

Type validation

- Ensure correct data types
- Convert where possible

Value validation

- Amounts ≥ 0
- Dates parseable

Logical validation

- `date_proposal <= date_approval <= date_completion`

Output:

- `valid_records`
 - `invalid_records`
 - `validation_report`
-

3.6 Data Cleaning

Standardize:

Null handling

Convert:

- `"N/A", "null", ""` → `None`

String normalization

- Lowercase
- Trim whitespace
- Remove duplicates spaces

Date normalization

- Convert all to ISO format

Numeric cleaning

- Remove commas, symbols
-

3.7 Corpus Object

Central abstraction:

```
class Corpus:
    records: List[Record]
    schema: Schema
    metadata: Dict
```

Must support:

- `.summary()`
 - `.head(n)`
 - `.describe()`
 - `.missing_report()`
-

4. Non-Functional Requirements

Performance

- Must handle **50k rows in < 5 seconds**

Reliability

- No crashes on bad data
- All errors logged

Determinism

- Synthetic data generation must support:

```
seed=42
```

5. Output Specification

After Stage 1, system must output:

5.1 Clean Dataset

- Fully normalized records

5.2 Validation Report

Example:

```
{
  "total_records": 20000,
  "valid_records": 16800,
  "invalid_records": 3200,
  "missing_fields": {
    "vendor_name": 12.3,
    "date_completion": 8.1
  },
}
```

```
"date_violations": 4.7  
}
```

5.3 Sample Inspection

```
corpus.head(5)  
corpus.summary()  
corpus.missing_report()
```

6. Acceptance Criteria

Stage is complete if:

- ☐ Synthetic dataset generated successfully
- ☐ Data loads without crashing
- ☐ Schema validation works
- ☐ Invalid records are detected
- ☐ Cleaning pipeline normalizes values
- ☐ Corpus object is usable downstream

7. Edge Cases (MANDATORY)

Handle:

- Completely empty dataset
- All-null column
- Single-record dataset
- Extremely large values
- Invalid date formats

8. Deliverables

- `data_generator.py`
- `ingestion.py`
- `schema.py`
- `cleaning.py`
- `corpus.py`

9. Example Usage

```
from parakh import Corpus, generate_dataset

# Generate synthetic data
df = generate_dataset(n=20000, seed=42)

# Load into system
corpus = Corpus.from_dataframe(df)

# Inspect
print(corpus.summary())
print(corpus.missing_report())
```

10. Definition of Done

- Pipeline runs end-to-end
- Produces clean, structured dataset
- Ready for Stage 2 (Confidence Layer)

11. Key Design Principle

Garbage in → controlled garbage out (not silent corruption)

This stage does NOT fix data completely.

It makes data **explicitly unreliable in a structured way**, enabling Stage 2 to quantify that unreliability.
